📘

Insertion Sort

🔹 1. What is Insertion Sort?

Insertion Sort builds the sorted array **one element at a time**.

👉 It takes one element and inserts it into its correct position among previously sorted elements.

It works similar to **sorting playing cards in hand**.

🔹 2. Real-Life Analogy

Imagine you are holding sorted cards:

```
3 7 9 12
```

Now you pick a new card:

```
5
```

You shift 7 and 9 and insert 5 in correct position:

```
3 5 7 9 12
```

That is exactly how insertion sort works.

🔹 3. Basic Idea

At step `i`:

- Assume first `i` elements are sorted.

- Insert `A[i]` into correct position in sorted part.

🔹 4. Pseudocode

```
INSERTION-SORT(A)
    for i = 1 to n-1
        key = A[i]
        j = i - 1

        while j >= 0 AND A[j] > key
            A[j+1] = A[j]
            j = j - 1

        A[j+1] = key
```

🔹 5. Clean C++ Code

```
#include <iostream>
#include <vector>
using namespace std;

void insertionSort(vector<int>& A) {
    int n = A.size();

    for (int i = 1; i < n; i++) {
        int key = A[i];      // element to insert
        int j = i - 1;

        // Shift elements greater than key to right
        while (j >= 0 && A[j] > key) {
            A[j + 1] = A[j];
            j--;
        }
```

```
        // Insert key at correct position
        A[j + 1] = key;
    }
}

void printArray(const vector<int>& A) {
    for (int x : A)
        cout << x << " ";
    cout << endl;
}

int main() {
    vector<int> arr = {8, 3, 5, 4, 7, 6, 1, 2};

    cout << "Before Sorting: ";
    printArray(arr);

    insertionSort(arr);

    cout << "After Sorting: ";
    printArray(arr);

    return 0;
}
```

🔹 6. Step-by-Step Example

Array:

```
8 3 5 4
```

Step 1 (i = 1)

key = 3

Compare with 8 → shift 8

Insert 3

```
3  8  5  4
```

Step 2 (i = 2)

key = 5

Compare with 8 → shift

Compare with 3 → stop

Insert 5

```
3  5  8  4
```

Step 3 (i = 3)

key = 4

Compare with 8 → shift

Compare with 5 → shift

Compare with 3 → stop

Insert 4

```
3  4  5  8
```

Sorted ✅

🔹 7. Time Complexity Analysis

Best Case (Already Sorted)

```
1  2  3  4  5
```

Only comparisons, no shifting.

$$T(n) = O(n)$$

Worst Case (Reverse Sorted)

```
5 4 3 2 1
```

Each element shifts all previous elements.

$T(n) = O(n^2)$

Average Case

$O(n^2)$

🔹 8. Space Complexity

- In-place sorting
- Extra space: **O(1)**

🔹 9. Properties

Property	Value
Stable	✅ Yes
In-place	✅ Yes
Adaptive	✅ Yes
Online	✅ Yes
Efficient for large data	❌ No

🔹 10. Why It Is Adaptive?

If array is nearly sorted:

Very few shifts happen.

Performance becomes close to **O(n)**.

🔹 11. Comparison with Bubble Sort

Feature	Bubble	Insertion
Swaps	Many	Fewer
Shifting	No	Yes
Best case	O(n)	O(n)
Practical use	Rare	Small arrays
Used inside other sorts	No	Yes (Hybrid sort)